



LICENCIATURA EN SISTEMAS

**DEPARTAMENTO DE DESARROLLO
PRODUCTIVO Y TECNOLÓGICO**

Programación de Sistemas ERP con Odoo

Guía de estudio y trabajos prácticos

Cuerpo Docente

Gustavo Siciliano, Javier Vescio

Versión: 20250927 (en desarrollo)

Año 2025

Material Práctico

Unidad N°1

Creación de módulo propio (modelos y vistas). Uso de pgweb. Seguridad con grupos.

Relaciones Many2one, One2many y Many2many.

1. Generá un entorno con Odoo 18 usando Doodba Copier Template.
2. Creá un módulo custom llamado “real_estate” con los archivos `__init__.py` y el `__manifest__.py` e instalalo.
3. Sobre el módulo anterior, creá un modelo “estate.property”, agregale los atributos listados a continuación y luego actualizá el módulo.

Nombre	Tipo	Atributos del campo
name	Char	string="Título", required=True
description	Text	string="Descripción"
postcode	Char	string="Código Postal"
date_availability	Date	string="Fecha disponibilidad"
expected_price	Float	string="Precio esperado"
selling_price	Float	string="Precio de venta"
bedrooms	Integer	string="Habitaciones", default=2
living_area	Integer	string="Superficie cubierta"
facades	Integer	string="Fachadas"
garage	Boolean	string="Garage"
garden	Boolean	string="Jardín"
garden_orientation	Selection	selection=[('north', 'Norte'), ('south', 'Sur'), ('east', 'Este'), ('west', 'Oeste'),], default="north", string="Orientación del jardín",

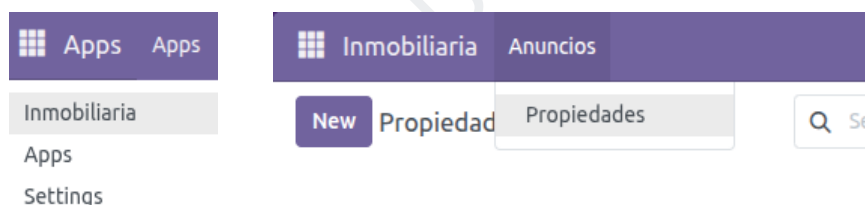
garden_area	Integer	string="Superficie jardín"
-------------	---------	----------------------------

Nota: recordá que los modelos se agregan dentro de la carpeta models y no olvides incluir el modelo en el init de models y en el init del módulo.

- Ingresá a pgweb (<http://localhost:18081>) y buscá el modelo que acabás de crear. Aparte de los campos que definiste, ¿qué otros campos adicionales se crearon?
- Creá un `estate_property_views.xml` en views y definí una acción (`ir.actions.act_window`) para el modelo que acabás de crear. Ponele como name “Propiedades” y “list, form” como `view_mode`.

Nota: el nombre `estate_property_views` es un nombre sugerido, pero podés ponerle el que quieras. No olvides incluir el archivo en el manifest.

- Creá un `real_estate_menuitem.xml` e incluí en él los tres niveles de menú (raíz, primer nivel y acción), conectando el menú del nivel de acción con la acción que creaste en el punto anterior. Al primer menú llámalo “Inmobiliaria”, al segundo “Anuncios” y al último “Propiedades”.



Incluí el archivo en el manifest debajo de la inclusión de “`estate_property_views.xml`” y actualizá el módulo. Aún no vas a poder ver los menús (recién en el siguiente ejercicio).

¿Qué pasa si en el manifest incluí a `real_estate_menuitem.xml` después de `estate_property_views.xml`? Probá.

- Aún no podés ver los menús que creaste en el ejercicio anterior (¿podés explicar por qué?). Para poder verlos, entrá en modo debug y convertite en SuperUsuario.

Nota: para salir del modo SuperUsuario, cerrá sesión y volvé a identificarte.

8. ¿Qué pasa si sólo tiene “list” el view_mode de la acción creada anteriormente? ¿Y si sólo es “form”? Probá ambos casos (recordá estar como SuperUsuario) y, al terminar, dejá “list, form” como estaba originalmente.
9. ¿Qué tipos de usuario existen en Odoo?
10. Creá un archivo `ir.model.access.csv` dentro de la carpeta `security` y hacé que el grupo “base.group_user” solamente pueda leer registros del modelo `estate.property`. Ahora, aunque no estés identificado/a como SuperUsuario, vas a poder ver los registros de `estate.property`.

¿A qué tipo de usuario representa el grupo “base.group_user”?
Si quisieras hacer que los usuarios de ese grupo puedan también crear, modificar y eliminar registros de ese modelo, ¿cómo harías?

Nota: no olvides incluir el archivo .csv en el manifest.
11. Desde Odoo, creá un grupo llamado “Manager de Propiedades”, asigná a tu usuario y dale permiso de lectura, creación, modificación y eliminación sobre el modelo “estate.property”. Probá ahora crear, modificar o eliminar registros.
12. Formalizá lo anterior creando el grupo desde tu módulo. Dentro de la carpeta `security`, creá un `real_estate_res_groups.xml` y definí el grupo “Manager de Propiedades” ahí mismo.

¿Qué ventajas y desventajas tiene hacer cambios directamente desde Odoo como hiciste en el punto anterior?
13. Dentro de `real_estate_res_groups.xml`, creá otro grupo llamado “Vendedor de Propiedades”.
14. En `ir.model.access.csv`, agregá una nueva línea para darle todos los permisos sobre el modelo `estate.property` al grupo Manager de Propiedades. Y la línea original que tenía el `base.group_user`, cambiale el grupo por el que creaste en el punto anterior.

15. ¿Qué pasa si un usuario no pertenece al grupo Vendedor ni Manager?
16. Creá una categoría de grupo llamada “Inmobiliaria” y vinculá los dos grupos que creaste con ella.

Nota: el objetivo es que los grupos que crea nuestro módulo no se “pierdan” entre los demás grupos que ya existen en Odoo. Ahora simplemente buscando el nombre de la categoría “Inmobiliaria” en la vista de grupos, nos aparecerán los grupos que creamos

☐	Group Name
☐	Inmobiliaria / Manager de Propiedades
☐	Inmobiliaria / Vendedor de Propiedades

17. Dentro de `estate_property_views.xml`, definí una vista de tipo search para que con los registros de `estate.property` se pueda:
- Buscar por name, postcode, expected_price, bedrooms, living_area y facades.
 - Filtrar fácilmente los registros para ver todos aquellos que hayan sido creados por el usuario que tiene la sesión activa.
 - Agrupar los registros por usuario que lo creó.
 - Agrupar los registros por mes.
 - Agrupar los registros por código postal.

Nota: aunque hay incontables ejemplos en código nativo, como el que se sugiere desde la [documentación oficial](#), podés apoyarte con la vista “view_account_invoice_report_search” del módulo Account.

18. Creá un registro de `estate.property` y luego desde la vista de lista, seleccioná el registro, hacé clic en acción y elegí “Duplicar”. Observá los resultados.
19. En los campos `date_availability` y `selling_price`, poné `copy = False`. Repetí el ejercicio anterior y observá los resultados. ¿Qué diferencias hay?

- 20.** Hacé que el campo `date_availability` tenga como valor por defecto la fecha actual incrementada en tres meses.

Nota: documentación sobre [today\(\)](#).

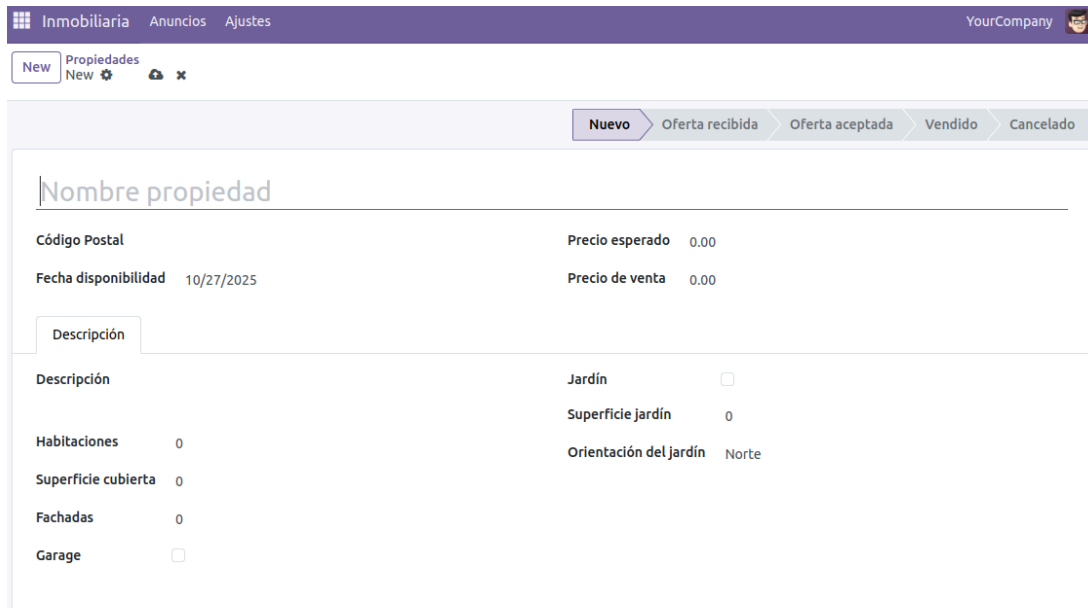
- 21.** Agregá el campo “state” en el modelo de propiedades con las siguientes opciones posibles: “Nuevo”, “Oferta recibida”, “Oferta aceptada”, “Vendido” y “Cancelado”. Además, hacé que sea un campo requerido, que su valor por defecto sea “Nuevo” y que su valor no se copie al hacer un duplicado del registro.

Nota: “state” es un [campo reservado](#) de tipo `selection`.

- 22.** La vista de lista de los registros de `estate.property` muestra solamente el nombre de cada propiedad. En `estate_property_views.xml`, creá una vista de lista “custom” vinculada al modelo en cuestión e incluí en ella los campos `name`, `postcode`, `bedrooms`, `living_area`, `expected_price`, `selling_price` y `date_availability`.

Nota: “custom” (personalizado) se refiere a algo creado por nosotros (algo no nativo); en este caso, una vista. Sin esa vista de lista custom, Odoo muestra por defecto una vista de lista que contiene sólo el nombre del registro. Si definís una vista de lista propia, Odoo la tomará automáticamente como la principal para ese modelo.

- 23.** Dentro de `estate_property_views.xml`, definí una nueva vista de formulario custom vinculada al modelo `estate.property`. Para el título (name) podés usar una etiqueta `H1` y un placeholder que diga “Nombre propiedad”. En un par de grupos para que queden dos columnas podés ubicar el código postal y fecha de disponibilidad y para la otra columna el precio esperado y el precio de venta. Para la notebook, en un grupo incluí los campos `description`, `bedrooms`, `living_area`, `facades`, `garage` y en otro grupo los campos `garden`, `garden_area` y `garden_orientation`. En el header del formulario incluí el campo `state` con el widget “statusbar”.



Nota: si no lográs que se vea igual o parecido, podés consultar el archivo [estate_property_views.xml](#), donde está todo el código completo hasta este ejercicio.

24. En `estate_property_views.xml`, sobre la vista de tipo `search`, permití que ahora se pueda:
 - a. Filtrar para mostrar todas las propiedades disponibles, es decir, cuyo estado sea `Nuevo` u `Oferta recibida`.
 - b. Agrupar los registros por estado.
25. Creá un modelo `"estate.property.type"` y agregale un campo `name` de tipo `Char` y que sea requerido.
26. Hacé que el `Manager de Propiedades` tenga permisos de acceso totales sobre los tipos de propiedad y que el `Vendedor de Propiedades` sólo pueda leer.
27. Creá un `estate_property_type_views.xml` en `views` y definí una acción para el modelo que acabás de crear. Ponele como `name` `"Tipos de propiedad"` y `"list, form"` como `view_mode`.
28. Dentro de `real_estate_menuitem.xml`, creá:

- a. Un menú de primer nivel llamado “Ajustes”, que debe situarse debajo del menú root que habías llamado “Inmobiliaria”.
- b. Un menú de acción, que debe situarse debajo del menú anterior y que conecte con la acción que definiste en el punto anterior.

Nota: si no podés ver los menús, no advertís error y actualizaste por consola, probá actualizando el módulo desde el propio Odoo. Y, en caso extremo, borra la caché del navegador web.



29. En `estate.property`, crear los siguientes campos many2one:

- a. “`property_type_id`” con título “Tipo Propiedad”, vinculado al modelo `estate.property.type`.
- b. “`buyer_id`” con título “Comprador”, vinculado al modelo `res.partner`.
- c. “`salesman_id`” con título “Vendedor”, vinculado al modelo `res.users`. Su valor no deberá ser copiado si se duplica el registro y el valor por defecto deberá ser el usuario logueado en el sistema.

Nota: podés ver este [ejemplo](#) propuesto por la doc. oficial para resolver el punto c.

30. En el formulario que definiste en `estate_property_views.xml`, agregá el campo “`property_type_id`” en el primer grupo, por encima del código postal. Creá, además, una nueva notebook que se llame “Más info.” y que muestre los campos del Comprador y Vendedor.

31. Creá un modelo “`estate.property.tag`” y agregale un campo `name` de tipo Char y que sea requerido. Aparte del `_name` propio de la clase, agregá un `_description` y poné “Etiqueta de propiedad”.

Nota 1: Aprovechá y poné un `_description` para `estate.property` y `estate.property.type`. Podría quedar bien “Propiedad” y “Tipo de propiedad” respectivamente.

Nota 2: La idea es que las propiedades puedan tener etiquetas (pequeñas leyendas) que las describan. Ejemplos: luminosa, espaciosa, minimalista, vista al río/mar, pet-friendly, acogedora, a estrenar, renovada, etc.

- 32.** Hacé que el Manager de Propiedades tenga permisos de acceso totales sobre las etiquetas de propiedad y que el Vendedor de Propiedades sólo pueda leer.
- 33.** Creá un `estate_property_tag_views.xml` en views y:
- Definí una acción para el modelo que acabás de crear. Ponele como name “Etiquetas de propiedad” y sólo “list” como `view_mode`.
 - Hacé una vista de lista custom que muestre el name del modelo de etiquetas y agregale el atributo `editable = “top”` a la lista (podés usar “bottom” también).
 - Replicá lo que hiciste en los puntos anteriores con el `estate.property.type`. Es decir, hacé los ajustes necesarios en `estate_property_type_views.xml`, para que sólo se muestra una vista de lista y se puedan crear los tipos de propiedad directamente desde la lista.

Nota: con lo hecho en el punto b, vas a poder crear registros desde la propia vista de lista sin usar la vista formulario. Esto es práctico cuando el modelo tiene uno o muy pocos campos.

- 34.** Dentro de `real_estate_menuitem.xml`, creá un menú de acción dentro del menú “Ajustes” que habías creado y conectalo con la acción que definiste para las etiquetas.
- 35.** En `estate.property`, agregá un campo `many2many` vinculado al modelo `estate.property.tag`. Llamá al campo `tag_ids` y como string ponele “Etiquetas”.
- ¿Por qué usamos un campo `many2many` y no un `many2one` o un `one2many`?

- 36.** En el formulario que definiste en `estate_property_views.xml`, agregá el campo `tag_ids` justo debajo del name. También agregá el campo en la vista de lista.

Nota: en ambos casos, como atributo del field, agregá widget="many2many_tags" ([ejemplo doc. oficial](#)). Fijate cómo se ve con y sin el widget.

37. Creá un modelo “estate.property.offer” y aparte del `_name` propio de la clase, agregá un `_description` y poné "Oferta sobre propiedad". Agregale los atributos listados a continuación y luego actualizá el módulo.

Nombre	Tipo	Atributos del campo
price	Float	string = “Precio”, required=True
status	Selection	string = “Estado”, selection=[(“accepted”, “Aceptada”), (“refused”, “Rechazada”)]
partner_id	Many2one	comodel = “res.partner”, string = “Ofertante”, required = True
property_id	Many2one	comodel = “estate.property”, string = “Propiedad”, required = True

38. Hacé que el Manager de Propiedades tenga permisos de acceso totales sobre las ofertas y que el Vendedor de Propiedades sólo pueda leer.
39. Agregá el campo `offer_ids` al modelo `estate.property` como una relación `one2many` de la siguiente manera:

```
offer_ids = fields.One2many(  
    comodel_name = "estate_property_offer",  
    inverse_name = "property_id",  
    string = "Ofertas"  
)
```

¿Por qué tenés que especificar un `inverse_name`?

¿Qué implica que la relación `One2many` sea `virtual`?

Nota: otra forma más abreviada es sin nombrar los atributos como `model_name` e `inverse_name`, es decir, `offer_ids = fields.One2many("estate_property_offer", "property_id", string = "Ofertas")`. Doc. oficial sobre relaciones [One2many](#).

40. Creá un `estate_property_offer_views.xml` en `views`, hacé una vista de lista custom con el atributo `editable = "top"` y agregá los campos `price`, `partner_id` y `status`. No es necesario que crees una acción... ¿por qué?
41. En `estate_property_views.xml`, sumá una nueva página llamada "Ofertas" y dentro de ella agregá el campo `offer_ids`.

Unidad N°2

Campos computados. Decoradores `api.depends`, `api.onchange`. Campos relacionados. Uso del ORM para operaciones CRUD. Constraints. Comandos relacionales. Herencia. Uso de `wdb` para depurar.

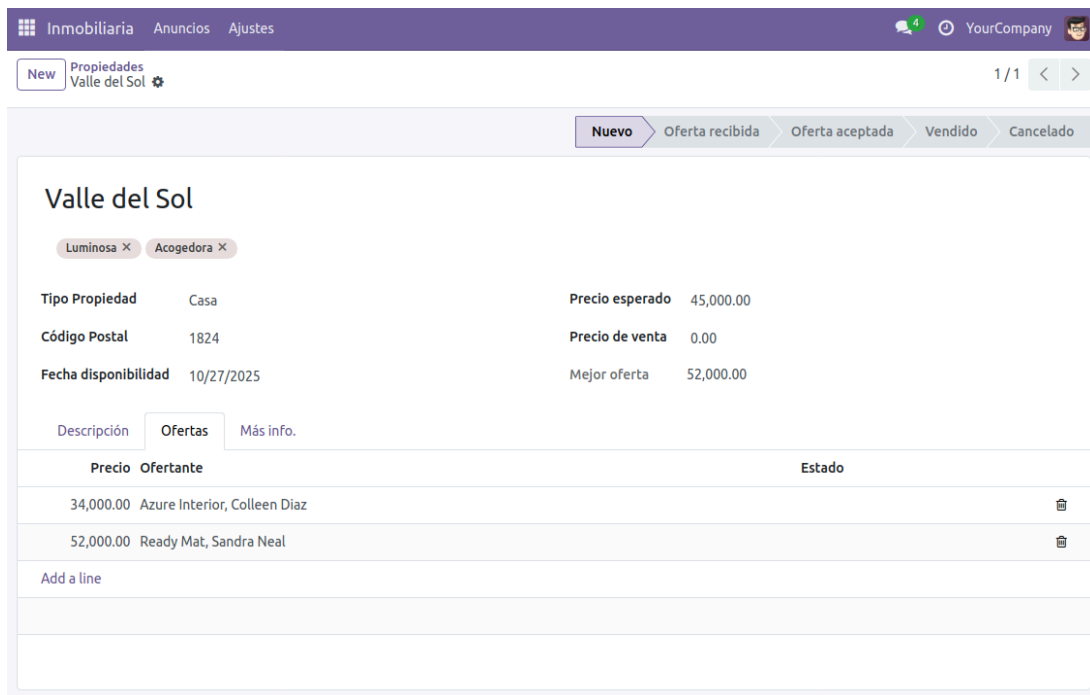
1. Sobre el modelo `estate.property`, creá un campo computado llamado `total_area`, cuyo valor se calcule automáticamente a partir de la suma de los valores ingresados en los campos `living_area` y `garden_area`. Como string ponelo “Superficie total”.

Nota: los [campos computados](#) invocan a un método que se encarga de hacer el cálculo y es buena práctica incluir el prefijo `_compute` como parte del nombre del método. Como el campo computado se llama “`total_area`”, es esperable que la función se llame `_compute_total_area`.

Por ahora, no uses el atributo `store = True`.

2. Agregá el campo anterior en la vista formulario de la propiedad, entre los campos presentes en la página “Descripción”. Luego, probá modificar los valores de `living_area` y `garden_area` y observá como automáticamente se calcula el valor de `total_area`.
3. Ingresá a pgweb y buscá el modelo `estate.property`. ¿Por qué no ves el campo en la tabla pero si en la vista formulario?
4. Hacé que el campo computado `total_area` sea almacenable, agregandole el atributo `store = True`. Actualizá la aplicación y probá nuevamente modificar `living_area` o `garden_area`. Vas a ver que, ahora, no se computará el campo `total_area`, por más que modifiques `living_area` y `garden_area`.
5. Agregá el decorador `@api.depends('living_area', 'garden_area')` a la función que computa, actualizá el módulo y volvé a probar modificar `living_area` o `garden_area`. ¡Sorpresa! Ahora el campo volverá a computarse. ¿Por qué?

6. Ingresá a una propiedad cualquiera que hayas creado (si aún no lo hiciste, creá una), entrá a la página de Ofertas y cargá al menos dos ofertantes con diferentes precios (no es necesario que les cargues el estado).
7. En el modelo estate.property, agregá un campo de tipo Float llamado best_offer con string “Mejor oferta”. Deberá ser un campo computado, que tomará el máximo valor de las ofertas que se hayan hecho sobre la propiedad. Mostrar el campo en la vista formulario debajo del precio de venta. ¿Almacenarías este campo computado?



The screenshot shows the Odoo interface for a property named 'Valle del Sol'. The page has a navigation bar with 'Inmobiliaria', 'Anuncios', and 'Ajustes'. Below the navigation bar, there's a 'New' button and a 'Propiedades' section. The main content area shows the property details: 'Tipo Propiedad: Casa', 'Código Postal: 1824', 'Fecha disponibilidad: 10/27/2025'. There are also tabs for 'Luminosa', 'Acogedora', 'Descripción', 'Ofertas', and 'Más info.'. The 'Ofertas' tab is active, showing a table with columns 'Precio', 'Ofertante', and 'Estado'. The table contains two rows of offers: one for 34,000.00 by 'Azure Interior, Colleen Diaz' and another for 52,000.00 by 'Ready Mat, Sandra Neal'. The 'Mejor oferta' (Best Offer) is highlighted as 52,000.00. The 'Precio esperado' (Expected Price) is 45,000.00 and the 'Precio de venta' (Selling Price) is 0.00.

Nota 1: podés obtener fácilmente una lista con todos los precios ofertados haciendo `offer_ids.mapped('price')`.

Nota 2: si no lográs resolver éste punto, podés consultar la [solución](#). ¡Tranquilo/a! Queda entre nosotros.

8. Como se va a volver un poco más compleja la carga de ofertas, en `estate_property_offer_views.xml` sacá el `editable = "top"` de la vista de lista. De esta forma, las ofertas se van a crear desde una vista formulario.

9. En el modelo de `estate.property.offer`, añadí los siguientes campos:
- `validity`, de tipo `Integer`, con default en 7 y string “Validez (días)”.
 - `date_deadline`, de tipo `Date` y string “Fecha límite”.

Luego, en la vista de lista vinculada a ese modelo, agregá los dos campos anteriores.

10. Hacé que el campo `date_deadline` se compute a partir de la suma del `create_date` y el `validity` ingresado por el usuario, pero permití también que el usuario pueda ingresar una fecha límite y que, si eso ocurre, que se calcule automáticamente el valor de la validez.

Nota: para realizar este punto, vas a tener que usar una [función inversa](#).

11. En el modelo de `estate.property.offer`, añadí como [campo relacionado](#) el tipo de propiedad y hacé que se almacene.
12. Creá una vista formulario con su menú correspondiente que muestre todas las ofertas. ¿Podés agrupar las ofertas por tipo de propiedad?
13. Creá un onchange para que cada vez que se tilde el campo booleano `garden` (Jardín) en el formulario de la propiedad, automáticamente el campo `garden_area` pase a tener como valor diez y que cada vez que se destilde el campo `garden`, se deje en cero a `garden_area`.
14. Dentro de un método ejecutado por un onchange, hacé que si el precio esperado es menor a diez mil se muestre un mensaje de advertencia no bloqueante informándole al usuario que el precio ingresado es bajo, por si se trató de un error de tipeo. Podés apoyarte en la [doc. oficial](#).

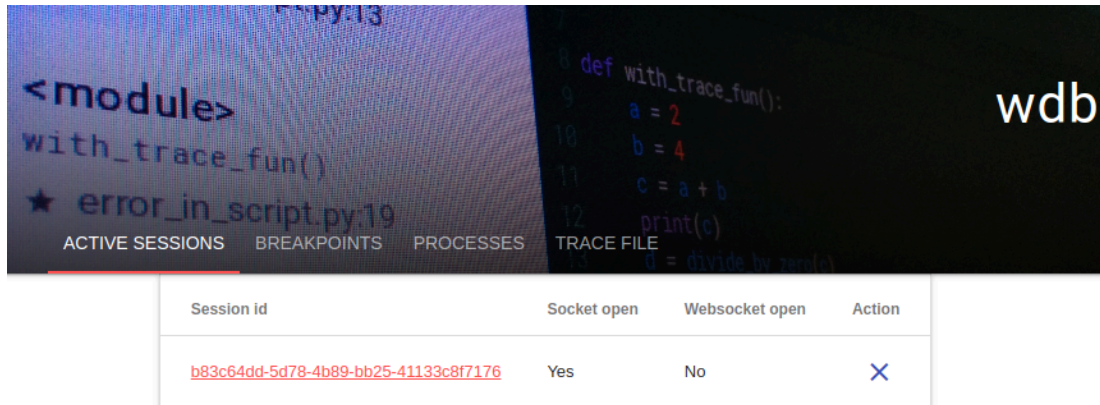
Nota: Podés hacer debugging del flujo de ejecución de las funciones implementadas importando `wdb` y haciendo un `set_trace()`. Por ejemplo:

```
@api.onchange('expected_price')
def _on_change_expected_price(self):
```

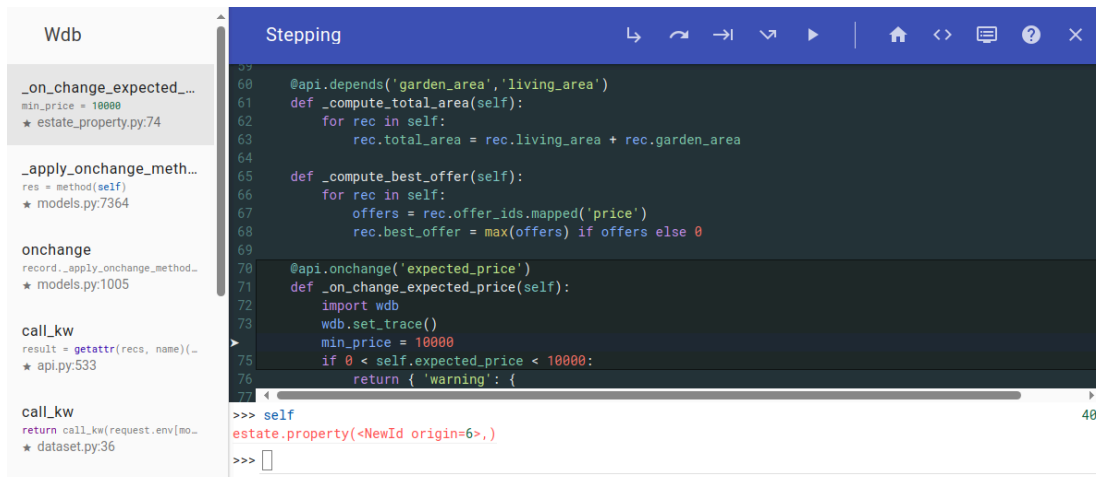
```
import wdb
wdb.set_trace()
```

Con lo anterior, el flujo de ejecución se interrumpirá al invocarse el método `_on_change_expected_price` debido al `wdb.set_trace()`.

Con el flujo interrumpido, si ingresás a WDB que incorpora Doodba (<http://localhost:18984>) vas a ver algo como esto:



Haciendo clic en el hash de la sesión, vas a poder ver el código, consultar y/o modificar variables, así como hacer uso de las funciones del ORM.



- En el header del formulario de las propiedades, agregá un par de botones; uno que diga “Cancelar” y otro “Marcar como vendida”. Asegurá que una propiedad cancelada no pueda ser marcada como vendida y viceversa lanzando una exception de tipo `UserError` si corresponde.

Si se marca una propiedad como vendida, mostrar un widget Ribbon que diga “Vendida”.

Nota: si bien es lo ideal tener una validación a nivel backend por las dudas, podés resolverlo a nivel frontend para mejorar la experiencia de usuario, haciendo que cada botón se muestre según el estado de la propiedad. Para ello, podés usar el atributo states.

16. Agregá un botón sobre la vista de lista de ofertas que te permita aceptar una oferta.

Si la oferta es aceptada, tenés que:

- a. Cargar el comprador y el precio de venta sobre la propiedad.
- b. Establecer en “oferta aceptada” el estado de la propiedad.
- c. Establecer automáticamente en rechazada cada una de las demás ofertas.

17. Realizá los siguientes puntos:

- a. Agregá un `sql_constraint` para garantizar que el nombre de las etiquetas sea único.
- b. Idem. anterior para los nombres de los tipos de propiedades.

El atributo `_sql_constraints` es una lista de tuplas, donde cada una de ellas se compone de un nombre, una condición y un mensaje de error a mostrar al usuario si la condición no se cumple.

Nota 1: asegurate de que las etiquetas (`estate.property.tag`) y tipos de propiedad (`estate.property.type`) que hayas creado no tengan el mismo nombre antes de realizar el ejercicio.

Nota 2: debajo del atributo `_description` del modelo `estate.property.tag`, podés definir lo siguiente:

```
_sql_constraints = [ ('unique_tag_name', 'UNIQUE(name)', 'El nombre de la etiqueta debe ser único') ]
```

18. Agregá un `sql_constraint` sobre `estate.property.offer` que asegure que una misma persona no pueda hacer más de una oferta sobre la misma propiedad.

Nota 1: desde la interfaz de usuario, borrará todas las ofertas que estén cargadas en cada una de las propiedades o asegurate de que cada propiedad no tenga más de una oferta de la misma persona antes de realizar el ejercicio.

Nota 2: tenés que hacer lo mismo que en el ejercicio anterior. Vas a tener que poner `UNIQUE(partner_id, property_id)`.

- 19.** Agregá en el modelo de propiedades un campo llamado `offer_partner_ids` que contenga todas las personas (`res.partner`) que hicieron ofertas sobre esa propiedad.

Nota: la idea es que a partir de las ofertas (campo `offer_ids`) obtengas los contactos. Con un campo computado vas a poder lograrlo. ¿Podrías usar un `related`?

- 20.** Agregá un botón en el encabezado del formulario de la propiedad que diga “Generar oferta automática”. Al presionarlo, deberá crearse automáticamente una nueva oferta sobre la propiedad que cumpla con los siguientes requerimientos:
- El precio ofertado deberá ser el precio esperado de la propiedad, con un incremento/decremento aleatorio dentro del rango -30% a +30%.
 - El ofertante deberá ser un contacto elegido al azar entre todos los contactos activos que aún no hayan realizado una oferta sobre la propiedad (es decir, que no estén incluidos en el campo `offer_partner_ids`). Para seleccionar uno de manera aleatoria, podés utilizar la función [`random.choice\(\)`](#).

- 21.** Agregá los siguientes botones en el encabezado de la propiedad:
- “Sacar etiquetas”: deberá desvincular de la propiedad todas las etiquetas cargadas.
 - “Cargar todas las etiquetas”: deberá vincular a la propiedad todas las etiquetas existentes.
 - “A estrenar”: deberá crear y vincular a la propiedad la etiqueta “A estrenar” si no existe la etiqueta y si existe deberá sólo vincularla.

*Nota: te recomendamos que leas la documentación que hay en esta misma guía sobre [comandos relacionales](#) y/o ver el video  **Comandos Relacionales** de la cátedra.*

 16: Herencia de métodos

22. Creá un método llamado `_unlink_if_new_or_cancelled` decorado con [`@api.ondetele\(\)`](#), para sólo permitir el borrado de propiedades cuyo estado sea “Nueva” o “Cancelada”.

¿Por qué es conveniente crear un método con el decorador `ondetele` en vez de redefinir el método `unlink()` y añadirle nuestra lógica de negocio?

23. Redefiní el método `create()` en el modelo `estate.property.offer` para que:
- Sólo pueda crearse una nueva oferta, si y sólo si el valor ofertado sobre la propiedad es mayor a la mejor oferta realizada hasta el momento. De no cumplirse la condición, mostrar un mensaje de error al usuario.
 - Sólo pueda crearse una nueva oferta, si y sólo si el estado de la propiedad es “Nuevo” u “Oferta recibida”.
 - Una vez creada una oferta, la propiedad cambie al estado “Oferta recibida”.

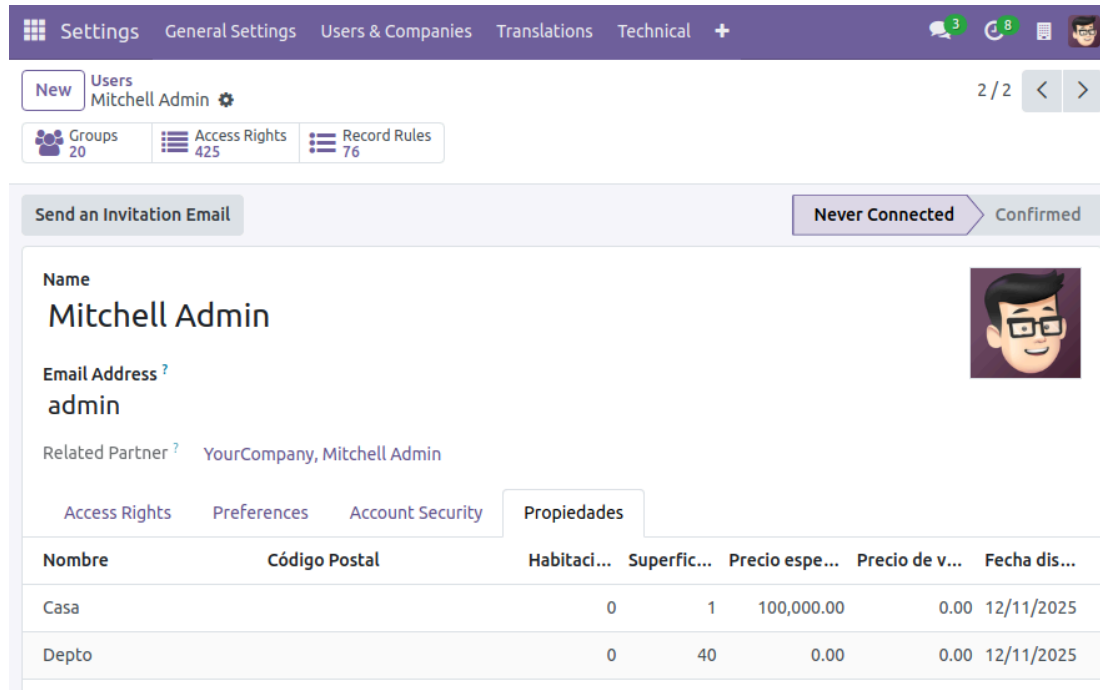
 17: Herencia de modelos y vistas

24. Creá un nuevo modelo que herede de “res.users” y agregale un campo One2many llamado `property_ids` vinculado inversamente con el campo Many2one `salesman_id` definido en `estate.property`.

```
class ResUsers(models.Model):  
    _inherit = "res.users"
```

Para este caso, ¿hay que definir reglas de acceso en el `ir.model.access.csv` como hicimos con `estate.property` y los demás modelos que creamos antes?

25. Creá un `res_users_views.xml` en views y hacé una herencia de vista para agregar una nueva página llamada “Propiedades” en la notebook que muestre el campo `property_ids` que creaste en el punto anterior. Debería verse como en la imagen:



Nota: podés evitar la creación de propiedades desde la nueva vista agregando un `readonly="1"` sobre el campo.

```
<field name="property_ids" readonly="1"/>
```

Facturación de ventas de propiedades |  18: Conexión entre módulos

26. Creá un módulo custom llamado “estate_account” con los archivos `__init__.py` y el `__manifest__.py`. En el manifest incluí como dependencias a los módulos `real_estate` y `account`. Instalá el módulo.
27. En el nuevo módulo, creá la carpeta `models` con un modelo nuevo que herede de “`estate.property`”. Redefiní la acción para “Marcar como vendida” una propiedad de forma tal que se genere una factura (modelo: `account.move`) considerando los siguientes puntos a la hora de crear el `account.move`:
- el `partner_id` deberá ser el comprador de la propiedad.

- b. el `move_type` deberá ser `"out_invoice"`, para indicar que es una factura a un cliente.
- c. en `line_ids` (modelo: `account.move.line`), deberá contener dos líneas de facturación con nombre, cantidad y precio unitario (campos técnicos `"name"`, `"quantity"` y `"price_unit"` respectivamente). La primera línea deberá tener el nombre de la propiedad, cantidad uno y precio de venta (`selling_price`) como precio unitario. La segunda línea deberá ser "Gastos administrativos", cantidad uno y cien como precio unitario.

Nota: usar comandos relaciones para hacer el punto b.

Unidad N°3

Creación de asistentes. QWeb. Generación de reportes. Creación de acciones planificadas.

Uso de MailHog. Pytest. Traducciones.

1. Creación de asistentes.
2. Generación de reportes.
3. Creación de acciones planificadas.
4. Uso de MailHog.
5. Pytest.
6. Traducciones.

UNLa

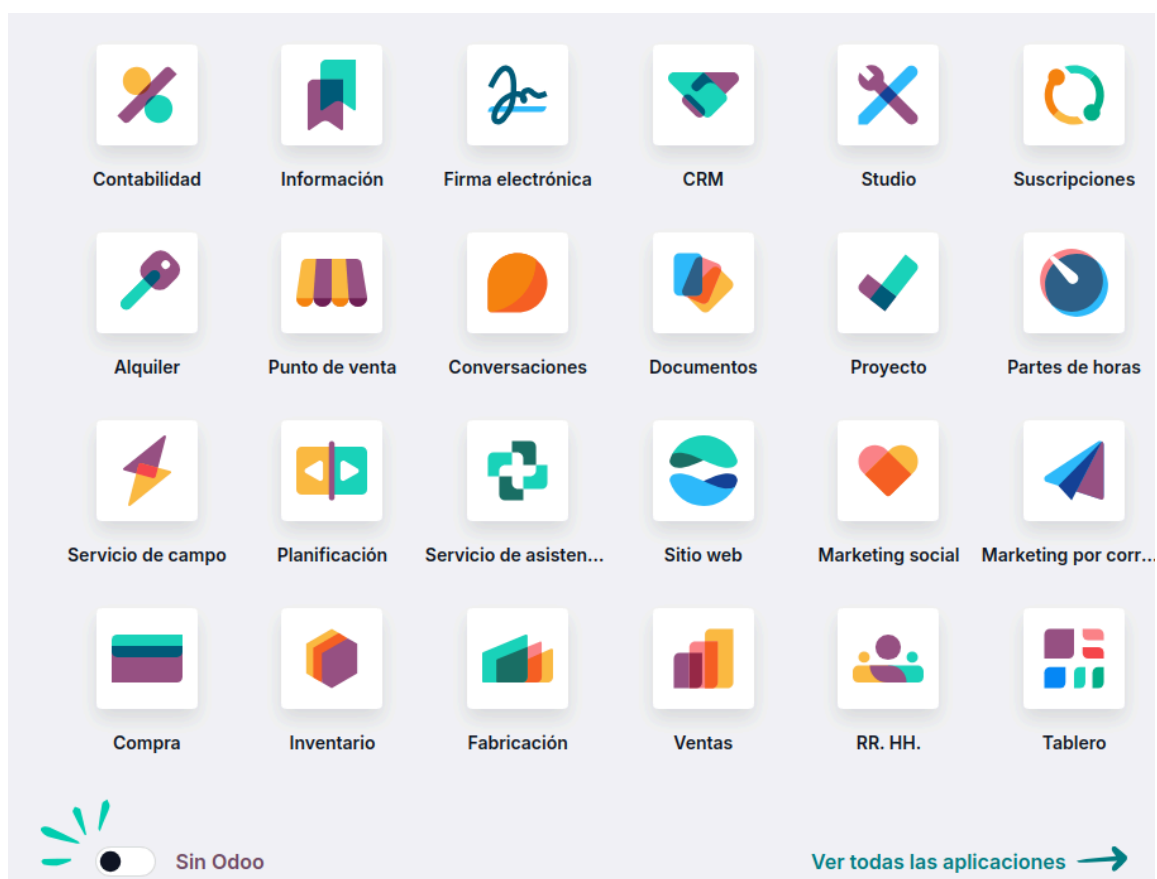
Material Teórico

¿Qué es un ERP?

ERP viene de Enterprise Resource Planning o Sistema de Planificación de Recursos empresariales. Básicamente un ERP es un sistema que te permite gestionar todas las áreas de una empresa de manera integral.

Toda la información se concentra en una única base de datos (información centralizada) y usás el mismo sistema ERP para gestionar cualquier aspecto que hace al día a día de una empresa.

Las aplicaciones no funcionan de manera separada, sino que se comunican entre sí. Por ejemplo, al registrar una venta en el módulo de Ventas, el sistema puede actualizar automáticamente el inventario y generar la factura en el módulo de Contabilidad.



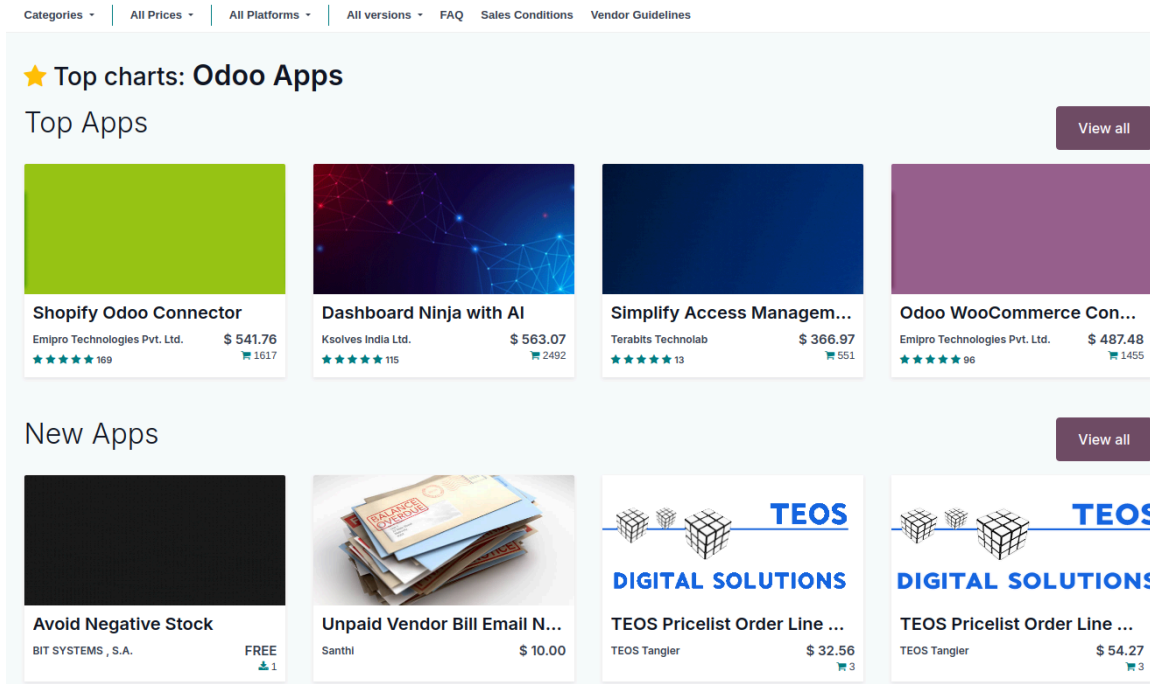
Por eso, el eslogan de Odoo es "todo tu negocio en una sola plataforma". Generalmente, acompañan ese lema con una imagen que muestra las principales aplicaciones incluidas

(Ventas, Compras, CRM, etc.), las cuales cubren la mayoría de las necesidades de cualquier empresa.

¿Qué es Odoo?

Odoo es un ERP de código abierto. Al estar escrito principalmente en Python, la curva de aprendizaje para desarrollar es razonable. Se puede desplegar en la nube o en servidores propios y cada año se publica una nueva versión mayor, con mejoras en usabilidad, nuevas apps y características para optimizar los procesos empresariales.

Existen otros ERP en el mercado como SAP, muy utilizado en grandes corporaciones con necesidades complejas, a diferencia de Odoo que pone más foco en pequeñas y medianas empresas.



The screenshot displays the Odoo App Store interface. At the top, there are navigation links: Categories, All Prices, All Platforms, All versions, FAQ, Sales Conditions, and Vendor Guidelines. Below this, the 'Top charts: Odoo Apps' section is highlighted with a star icon. Underneath, the 'Top Apps' section lists four applications with their respective icons, names, developers, prices, and ratings. A 'View all' button is located to the right of this section. The 'New Apps' section follows, also with a 'View all' button. It lists four new applications, including 'Avoid Negative Stock' (free), 'Unpaid Vendor Bill Email N...', and two versions of 'TEOS Digital Solutions' (priced at \$32.56 and \$54.27).

App Name	Developer	Price	Rating
Shopify Odoo Connector	Emipro Technologies Pvt. Ltd.	\$ 541.76	★★★★★ 169
Dashboard Ninja with AI	Ksolves India Ltd.	\$ 563.07	★★★★★ 115
Simplify Access Managem...	Terabits Technolab	\$ 366.97	★★★★★ 13
Odoo WooCommerce Con...	Emipro Technologies Pvt. Ltd.	\$ 487.48	★★★★★ 96
Avoid Negative Stock	BIT SYSTEMS, S.A.	FREE	★ 1
Unpaid Vendor Bill Email N...	Santhi	\$ 10.00	
TEOS Digital Solutions	TEOS Tangler	\$ 32.56	★ 3
TEOS Digital Solutions	TEOS Tangler	\$ 54.27	★ 3

Odoo es altamente configurable y permite fácilmente modificar los procesos funcionales. Se pueden obtener nuevas aplicaciones desde el App Store de Odoo, nuevas aplicaciones de terceros o incluso desarrollar una propia. Existen miles de aplicaciones desarrolladas por la comunidad y por la empresa creadora de Odoo, lo que permite cubrir prácticamente todas las áreas de una organización.

En resumen... Un ERP integra y centraliza toda la información de una empresa. Dentro los ERP tenemos a Odoo, que es de código abierto y escrito principalmente en Python. Es modular, agiliza procesos y permite crecer en funcionalidades según las necesidades de cada empresa.

Campo Relacionado (related)

Es un campo cuyo valor se obtiene directamente a partir de otro campo de un modelo relacionado. Por defecto no se almacena en la base de datos, sino que se computa cuando uno lo muestra en alguna vista, aunque se puede utilizar el atributo `store = True` para que se almacene. Por defecto es de sólo lectura. Ejemplo:

```
partner_id = fields.Many2one('res.partner', string='Ofertante')

email = fields.Char(related='partner_id.email')
```

Notá que el email se obtiene del `partner_id`. Es importante que el tipo de dato coincida con el tipo de dato real del campo relacionado.

Los campos relacionados son un tipo especial y simplificado de campo computado.

Campo Computado

Es un campo cuyo valor surge de hacer alguna operación. Por defecto, un campo computado no se guarda en la base de datos: Odoo ejecuta su método de cálculo cada vez que una vista lo necesita. El siguiente ejemplo reproduce el mismo resultado que el mostrado en Campos Relacionados, pero implementado como un campo computado:

```
partner_id = fields.Many2one('res.partner', string='Ofertante')

email = fields.Char(compute='_compute_email')

def _compute_email(self):

    for rec in self:

        rec.email = rec.partner_id.email
```


Cada vez que abrís una vista que muestra ese campo, se llama a la función que especificaste en el compute. Si es una vista formulario, el cómputo se hará una vez porque el self contendrá un único registro (ya que el formulario sólo muestra un único registro), pero si es una vista de lista el self contendrá los registros que se están visualizando y, en consecuencia, habrá una iteración (o cómputo) por cada registro.

Poniendo `store = True` como atributo del campo, el valor del mismo se calcula una sólo vez, se almacena en la tabla vinculada al modelo y Odoo lo lee directamente de la base de datos como si fuera cualquier otro campo normal. La gracia de usar `store = True` es como decir: “voy a almacenar el valor ya calculado, así la próxima vez que necesito tomar el valor del campo, no tengo que volver a calcularlo, simplemente leo el valor que está en la base y me ahorro el costo en tiempo y espacio de hacer el cómputo”.

Si decidís que el campo computado se almacene en base, tenés que usar el decorador `@api.depends(...)` para indicarle a Odoo otros campos que, de actualizarse, deberán disparar nuevamente el cálculo del campo computado.

Tené en cuenta que, en algunos casos, ese recomputo puede llegar a darse sobre muchísimos registros (quizás miles), lo que podría tener un impacto muy negativo sobre el rendimiento del sistema. En general esto se da cuando la dependencia que origina el recomputo del campo, hace referencia a otro campo que no es propio del modelo en sí, sino de un registro perteneciente a otro modelo. Podés tomar como ejemplo el siguiente código que muestran en el [capítulo 8 del tutorial Server Framework 101](#). Si el name de cierto partner_id cambia, automáticamente eso provocaría el recomputo del campo description de todos los registros que tengan ese mismo partner_id.

```
description = fields.Char(compute="_compute_description", store=True)
partner_id = fields.Many2one("res.partner")

@api.depends("partner_id.name")
def _compute_description(self):
    for record in self:
        record.description = "Hola %s" % record.partner_id.name
```

¿Almacenar o no almacenar? Esa es la cuestión.

- Si el campo computado solamente lo mostrás en la vista formulario, podés prescindir su almacenado en base.
- Si querés que se puedan buscar registros del modelo a partir del campo computado y te preocupa el rendimiento, podés seguir prescindiendo del almacenado y definir un método [_search](#); si considerás que el rendimiento no es un problema, almacenalo.
- Si querés que el campo computado se muestre en la vista de lista, considerá almacenarlo porque, en una vista de lista, se pueden visualizar hasta cientos de registros a la vez.

El campo computado es de sólo lectura, pero hay un caso especial en el que puede dejar de serlo y es cuando tenemos un cómputo inverso. Considerá el siguiente caso de ejemplo: suponé que tenés un campo de tipo entero llamado “numero” y otro campo computado llamado “doble_de_numero” que se calcula multiplicando por dos el valor del campo entero. Puede que quieras definir un cómputo inverso, de forma tal que si se ingresa un entero cualquiera en el campo computado, el campo “numero” pase a valer el entero dividido dos. Un cómputo inverso se define también con una función sobre el campo computado y la sólo definición hace que el campo deje de ser de sólo lectura.

```
numero = fields.Integer(string='Número')

doble_de_numero = fields.Integer(

    string='Doble de número', compute='_compute_doble_de_numero',

    inverse='_inverse_doble_de_numero'

)

def _compute_doble_de_numero(self):

    for rec in self:

        rec.doble_de_numero = rec.numero * 2

def _inverse_doble_de_numero(self):

    for rec in self:

        rec.numero = rec.doble_de_numero / 2
```

Comandos Relacionales

La manipulación de registros presentes en campos One2many y Many2many se realiza mediante el uso de comandos relacionales. Con ellos, vas a poder crear, actualizar, eliminar o vincular registros relacionados. Internamente cada comando es una [tupla](#) de tres elementos:

- Número entero que identifica el comando.
- Id del registro (si aplica).
- Diccionario con los datos del registro a cargar/actualizar (si aplica).

Y en total hay siete comandos posibles:

CREATE= 0

UPDATE= 1

DELETE= 2

UNLINK= 3

LINK= 4

CLEAR= 5

SET= 6

Si quisieras actualizar un registro existente de cierto 'id' con los nuevos key-values definidos en un diccionario 'dic', el comando relacional debería ser:

(1, id, dic)

Ejemplo:

```
(1, line.id, {  
    'balance': -line.balance,  
    'amount_currency': -line.amount_currency  
}),
```

Para no tener que poner un número entero por cada operación y facilitar la lectura, Odoo recomienda usar funciones que te retornan la tupla directamente. Basta con importar Command de odoo (`from odoo import Command`) para conseguir el mismo resultado anterior usando Command.update:

```
Command.update(line.id, {
    'balance': -line.balance,
    'amount_currency': -line.amount_currency
})
```

Las funciones son:

```
Command.create({values})      # equivale a (0, 0, {values})
Command.update(id, {values})  # equivale a (1, id, {values})
Command.delete(id)           # equivale a (2, id, 0)
Command.unlink(id)           # equivale a (3, id, 0)
Command.link(id)             # equivale a (4, id, 0)
Command.clear()              # equivale a (5, 0, 0)
Command.set([ids])           # equivale a (6, 0, ids)
```

Considerá, por ejemplo, el siguiente modelo:

```
class EstateProperty(models.Model):
    _name = 'estate.property'

    tag_ids = fields.Many2many(
        'estate.property.tag', string='Etiquetas'
    )
```

Si querés manipular `tag_ids`, simplemente asigne una lista con los comandos relacionales que sean de tu necesidad.

Por ejemplo, podrías hacer algo como:

```
tag_ids = [Command.create({'name': 'A estrenar'})]
```

Eso crearía un registro en `estate.property.tag` con el name 'A estrenar' y además vincularía al mismo en `tag_ids`. Con tupla explícita se vería así:

```
tag_ids = [(0, 0, {'name': 'A estrenar'})]
```

Mirá éste otro ejemplo:

```
tag_ids = [  
    Command.create({'name': 'Vista a la costa'}),  
    Command.create({'name': 'Luminosa'})  
]
```

Lo anterior crearía y vincularía dos registros más (aparte del ya existente ‘A estrenar’), consiguiendo así tener tres etiquetas en total.

Con `tag_ids = [Command.unlink(id_tag)]` desvincularías al `estate.property.tag(id_tag)` de `tag_ids`, pero el registro de la etiqueta seguiría existiendo en `estate.property.tag` (distinto sería el caso si usaras el comando `delete`, donde aparte de romperse el vínculo, la etiqueta se borraría de `estate.property.tag`). Asimismo, con el comando “`link(id_tag)`” podés establecer un vínculo. Es decir, podrías vincular una etiqueta existente de cierto `id_tag` con tu `estate.property`.

Con `clear()` borrarías todos los vínculos y con `set([id1, id2, ..., idn])` podrías vincular uno o varios registros con los ids de etiquetas especificados a las propiedad.

Más información en la [doc. oficial](#).

1 - Fundamentación de la Asignatura:

Odoo es un ERP de código abierto con creciente presencia en empresas de la región. Su arquitectura modular permite a los desarrolladores extender sus funcionalidades con relativa facilidad. Esta asignatura introduce a los estudiantes en la personalización y el desarrollo de módulos sobre Odoo, complementando los conocimientos de sistemas y organizaciones, bases de datos y programación orientada a objetos adquiridos en la carrera.

2 - Objetivos:

- Analizar un requerimiento simple de negocio y decidir si se resuelve con configuración o con desarrollo sobre Odoo.
- Modelar datos utilizando el ORM de Odoo y sus tipos de relación.
- Implementar vistas, lógica de negocio básica y reportes en un módulo propio.
- Tener una instancia Odoo en cualquier versión con contenedores Docker usando Doodba.

3 - Contenidos Mínimos:

Concepto de ERP. Odoo versión Community vs Enterprise.
Anatomía de un módulo.
Instalación y gestión de entornos de desarrollo Odoo con Doodba.
Creación de módulo propio (modelos y vistas).
Herencia de modelos.
Seguridad con grupos.
Campos computados.
Relaciones.
Uso del ORM para operaciones CRUD.
Creación de asistentes.

4 - Contenidos:

UNIDAD 1:

Concepto de ERP. Odoo versión Community vs Enterprise. Tecnologías usadas. Anatomía de un módulo. Instalación y gestión de entornos de desarrollo Odoo con Doodba. Exploración de módulos nativos. Uso de pgweb. Creación de módulo propio (modelos y vistas). Seguridad con grupos. Relaciones Many2one, One2many y Many2many.

UNIDAD 2:

Campos computados. Decoradores `api.depends`, `api.onchange`. Campos relacionados. Uso del ORM para operaciones CRUD. Constraints. Comandos relacionales. Herencia. Uso de `wdb` para depurar.

UNIDAD 3:

Creación de asistentes. QWeb. Generación de reportes. Creación de acciones planificadas. Uso de MailHog. Pytest. Traducciones.

5 - Metodología de Trabajo:

Las clases serán teórico-prácticas: cada sesión empezará con la explicación del tema y continuará con una demostración. Para poder seguir las actividades y entregar los trabajos, el alumnado deberá disponer de un entorno de Odoo 18 (o superior).

6- Desarrollo de Actividades Prácticas

Desde un principio, al final de cada clase se dejará tarea a los estudiantes que deberán hacer en grupo y subirla a GitHub. Deberán resolver un trabajo práctico cuatrimestral también de manera grupal. Además, durante la cursada se realizarán prácticas en conjunto entre el docente y los estudiantes.

7 - Evaluación y Acreditación:

- El estudiante deberá cumplir con una asistencia mínima del 75% sobre el total de clases presenciales.
- En las clases virtuales no se registrará asistencia; sin embargo, el estudiante deberá haber entregado en tiempo y forma al menos el 75% de las tareas asignadas en GitHub para mantener la condición de regular.
- Será requisito aprobar un trabajo práctico cuatrimestral. En caso de desaprobado dicho trabajo, el estudiante contará con una instancia de recuperación en la fecha correspondiente, posterior a la entrega de la nota. Si, luego de esta instancia, el trabajo práctico continuara desaprobado, el estudiante perderá la regularidad.

8 -Bibliografía:

- Doodba Copier Template (<https://github.com/Tecnativa/doodba-copier-template>)
- Server Framework 101
(https://www.odoo.com/documentation/18.0/developer/tutorials/server_framework_101.htm)
- Technical Training - Introduction to Development
(<https://www.odoo.com/slides/technical-training-introduction-to-development-318>)
- Documentación oficial (<https://www.odoo.com/documentation>)
- Aprende a desarrollar en Odoo - Curso Técnico
(<https://www.youtube.com/playlist?list=PL1-aSABtP6ABaEl9nfhPnb7Dss4Hv2ELS>)